

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..27

CUSTOMIZATIONS

 Copy as Markdown

Customize Microsoft sign-in

This page covers configuration options for the Microsoft identity provider beyond the basic setup, on both the server and the app. It also shows how to customize the sign-in UI. You can use the `MicrosoftSignInWidget` to display the Microsoft sign-in flow in your own custom UI, or the `MicrosoftAuthController` to build a completely custom authentication interface.

Server configuration

These options are set on the `MicrosoftIdpConfig` in your server code.

Configuration options

Below is a non-exhaustive list of some of the most common configuration options. For more details on all options, check the `MicrosoftIdpConfig` in-code documentation.

Tenant configuration

The `tenant` parameter determines which accounts can sign in to your application:

- `'common'` (default) - Allows both personal Microsoft accounts and work/school accounts.
- `'organizations'` - Allows only work/school accounts (any organization).
- `'consumers'` - Allows only personal Microsoft accounts.
- A specific tenant ID - Restricts to accounts from a specific Microsoft Entra ID tenant.

```
final microsoftIdpConfig = MicrosoftIdpConfig(  
  clientId: pod.getPassword('microsoftClientId')!,  
  clientSecret: pod.getPassword('microsoftClientSecret')!,  
  tenant: 'organizations', // Only allow work/school accounts  
);
```



TIP

Use `'common'` for the widest user base. Use a specific tenant ID when building internal applications for a single organization.

Custom account validation

You can customize the validation for Microsoft account details before allowing sign-in. By default, the validation checks that the received account details contain a non-empty `userIdentifier`.

```
final microsoftIdpConfig = MicrosoftIdpConfigFromPasswords(  
  // Optional: Custom validation for Microsoft account details  
  microsoftAccountDetailsValidation: (MicrosoftAccountDetails accountDetails) {  
    // Throw an exception if account doesn't meet custom requirements  
    if (accountDetails.userIdentifier.isEmpty) {  
      throw MicrosoftUserInfoMissingDataException();  
    }  
    // Example: Require email to be present  
    if (accountDetails.email == null || accountDetails.email!.isEmpty) {  
      throw MicrosoftUserInfoMissingDataException();  
    }  
  },  
);
```

NOTE

Users may choose not to share their email or other information during the Microsoft login flow. Adjust your validation function carefully to avoid blocking legitimate users.

MicrosoftAccountDetails

The `microsoftAccountDetailsValidation` callback receives a `MicrosoftAccountDetails` record with the following properties:

Property	Type	Description
<code>userIdentifier</code>	<code>String</code>	The Microsoft user's unique identifier (Object ID)
<code>email</code>	<code>String?</code>	The user's email address (may be null)

Property	Type	Description
<code>name</code>	<code>String?</code>	The user's display name from Microsoft
<code>imageBytes</code>	<code>Uint8List?</code>	The user's profile photo. Always <code>null</code> during validation, because the photo is fetched afterwards, and only when the <code>fetchProfilePhoto</code> option on <code>MicrosoftIdpConfig</code> is enabled (the default)

Example of accessing these properties:

```
microsoftAccountDetailsValidation: (accountDetails) {
  print('Microsoft Object ID: ${accountDetails.userIdentifier}');
  print('Email: ${accountDetails.email}');
  print('Display name: ${accountDetails.name}');
  // imageBytes is always null here. The photo is fetched after validation.

  // Custom validation logic
  if (accountDetails.email == null) {
    throw MicrosoftUserInfoMissingDataException();
  }
},
```

INFO

The properties available depend on the scopes requested and what the user consented to share.

Accessing Microsoft APIs on the server

CAUTION

The `getExtraMicrosoftInfoCallback` below runs on **every** sign-in, not only the first. Cache what you fetch, and guard external calls with `try/catch` so a provider outage does not block sign-in.

On the server side, you can access Microsoft APIs using the access token. The `getExtraMicrosoftInfoCallback` in `MicrosoftIdpConfig` receives the access token and can be used to call Microsoft Graph APIs:

```
import 'package:http/http.dart' as http;

final microsoftIdpConfig = MicrosoftIdpConfigFromPasswords(
  // Optional: Extract additional info from Microsoft Graph APIs
```

```

getExtraMicrosoftInfoCallback: (session, {
    required accountDetails,
    required accessToken,
    required transaction,
}) async {
    // Use accessToken to call Microsoft Graph APIs and store additional info
    // Example: Access user's calendar
    final response = await http.get(
        Uri.https('graph.microsoft.com', '/v1.0/me/calendar'),
        headers: {'Authorization': 'Bearer $accessToken'},
    );
    // Process response and store additional info in the database
},
);

```

Reacting to account creation

You can use the `onAfterMicrosoftAccountCreated` callback to run logic after a new Microsoft account has been created and linked to an auth user. This callback is only invoked for new accounts, not for returning users.

This callback is complementary to the core `onAfterAuthUserCreated` callback. Use it for side effects specific to a Microsoft login, like storing analytics, sending a welcome email, or storing additional data.

```

final microsoftIdpConfig = MicrosoftIdpConfigFromPasswords(
    onAfterMicrosoftAccountCreated: (
        session,
        authUser,
        microsoftAccount, {
            required transaction,
        }) async {
        // e.g. store additional data, send a welcome email, or log for analytics
    },
);

```

INFO

This callback runs inside the same database transaction as the account creation. Throwing an exception inside this callback will abort the process. If you perform external side-effects, make sure to safeguard them with a try/catch to prevent unwanted failures.

CAUTION

Scopes you assign here with `AuthServices.instance.authUsers.update()` do not apply to the login that is already in progress, because token issuance uses the scopes loaded before this callback runs. They take effect the next time the user signs in. To assign scopes at creation time instead, use

`onBeforeAuthUserCreated` together with `getExtraMicrosoftInfoCallback`, which runs before the auth user is created.

App configuration

These options configure Microsoft sign-in in your Flutter app.

Requesting additional Microsoft scopes

The default setup allows access to basic user information, such as `name`, `email`. You may require additional access scopes to access other Microsoft APIs, such as accessing a user's calendar, mail, or OneDrive files.

The default scopes requested are:

- `openid`: Required for OpenID Connect authentication.
- `profile`: Access to user's basic profile information.
- `email`: Access to user's email address.
- `offline_access`: Allows refresh tokens for long-lived sessions.
- `https://graph.microsoft.com/User.Read`: Access to user's Microsoft Graph profile.

To request additional scopes, you will need to:

- Ensure the required API permissions are configured in your Microsoft Entra ID app registration (navigate to **API permissions** in the [Azure Portal](#)).
- Request access to the scopes when signing in. Do this by setting the `scopes` parameter of the `MicrosoftSignInWidget` or `MicrosoftAuthController`.

A full list of available scopes and Microsoft Graph API permissions can be found in the [Microsoft Graph permissions reference](#).

INFO

Adding additional scopes may require admin consent depending on your tenant configuration and the sensitivity of the requested permissions.

To use the granted scopes from the server with the access token, see [Accessing Microsoft APIs on the server](#).

Configuring client IDs on the app

Passing client IDs in code

You can pass the `clientId`, `redirectUri`, and `tenant` directly when initializing the Microsoft sign-in service:

```
await client.auth.initializeMicrosoftSignIn(  
  clientId: 'YOUR_MICROSOFT_CLIENT_ID',  
  redirectUri: 'yourapp://auth',  
  tenant: 'common', // Optional, defaults to 'common'  
);
```

This approach is useful when you need different client IDs per platform and want to manage them in your Dart code.

Using environment variables

Alternatively, you can pass client configuration during build time using the `--dart-define` option. The Microsoft sign-in provider supports the following environment variables:

- `MICROSOFT_CLIENT_ID`: Your Microsoft Application (client) ID
- `MICROSOFT_REDIRECT_URI`: The callback URI

Example usage:

```
flutter run -d <device> \  
  --dart-define="MICROSOFT_CLIENT_ID=your_client_id" \  
  --dart-define="MICROSOFT_REDIRECT_URI=msauth://auth"
```

The tenant has no environment variable. Pass it as an argument when you initialize Microsoft sign-in.

This approach is useful when you need to:

- Manage separate client IDs for different platforms (Android, iOS, web, macOS) in a centralized way
- Avoid committing client IDs to version control
- Configure different credentials for different build environments (development, staging, production)

TIP

You can also set these environment variables in your IDE's run configuration or CI/CD pipeline to avoid passing them manually each time.

Customize the sign-in button

Inside `SignInWidget`, fields set on its `buttonStyle` take precedence over the settings below, as described in [Styling the buttons](#).

INFO

The `SignInWidget` uses the `MicrosoftSignInWidget` internally to display the Microsoft sign-in flow. You can also supply a custom `MicrosoftSignInWidget` to the `SignInWidget` to override the default behavior.

```
SignInWidget(  
  client: client,  
  microsoftSignInWidget: MicrosoftSignInWidget(  
    client: client,  
    // Shape and label survive inside SignInWidget, unless its buttonStyle  
    // sets them. Brand colors do not.  
    shape: SignInButtonShape.rounded,  
    text: SignInButtonTextVariant.signInWith,  
    // A custom widget replaces the built-in handling, so pass your own callbacks.  
    onAuthenticated: () { /* ... */ },  
    onError: (error) { /* ... */ },  
  ),  
)
```

Using the `MicrosoftSignInWidget`

The `MicrosoftSignInWidget` handles the complete Microsoft sign-in flow for your Flutter app.

You can customize the widget's appearance and behavior:

```
MicrosoftSignInWidget(  
  client: client,  
  // Button customization. The values shown are the defaults.  
  style: MicrosoftButtonStyle.light, // or dark  
  size: SignInButtonSize.large, // or medium, small  
  text: SignInButtonTextVariant.continueWith, // or signInWith, signUpWith, signIn  
  shape: SignInButtonShape.pill, // or rounded, rectangular  
  logoAlignment: SignInButtonLogoAlignment.center, // or left  
  minimumWidth: 240, // at most 400  
  textStyle: null, // TextStyle for the label  
  
  // Scopes to request from Microsoft  
  // These are the default scopes.  
  scopes: const [  
    'openid',  
    'profile',  
    'email',  
    'offline_access',  
    'https://graph.microsoft.com/User.Read',  
  ],  
)
```

```

onAuthenticated: () {
  // Do something when the user is authenticated.
  //
  // NOTE: You should not navigate to the home screen here, otherwise
  // the user will have to sign in again every time they open the app.
},
onError: (error) {
  // Handle errors
},
)

```

Build a custom UI with MicrosoftAuthController

For more control over the UI, you can use the `MicrosoftAuthController` class, which provides all the authentication logic without any UI components. This allows you to build a completely custom authentication interface.

```

import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';

final controller = MicrosoftAuthController(
  client: client,
  onAuthenticated: () {
    // Do something when the user is authenticated.
    //
    // NOTE: You should not navigate to the home screen here, otherwise
    // the user will have to sign in again every time they open the app.
  },
  onError: (error) {
    // Handle errors
  },
  scopes: const [
    'openid',
    'profile',
    'email',
    'offline_access',
    'https://graph.microsoft.com/User.Read',
  ],
);

// Initiate sign-in
await controller.signIn();

```

MicrosoftAuthController state management

Your widget should render the appropriate UI based on the `state` property of the controller. You can also use the below state properties to build your UI:

```

// Check current state
final state = controller.state; // MicrosoftAuthState enum

// Check if loading

```

```
final isLoading = controller.isLoading;

// Check if authenticated
final isAuthenticated = controller.isAuthenticated;

// Get error message
final errorMessage = controller.errorMessage;

// Listen to state changes
controller.addListener(() {
  setState(() {
    // Rebuild UI when controller state changes
  });
});
```

MicrosoftAuthController states

- `MicrosoftAuthState.idle` - Ready for user interaction.
- `MicrosoftAuthState.loading` - Processing a sign-in request.
- `MicrosoftAuthState.error` - An error occurred.
- `MicrosoftAuthState.authenticated` - Authentication was successful.

Related

- [Setup](#): set up Microsoft sign-in on the server and in your app.
- [Troubleshooting](#): fix common Microsoft sign-in errors.
- [UI components](#): use and style the built-in sign-in UI.
- [Working with users](#): manage auth users and react to user events.